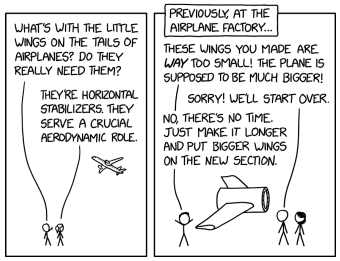
But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Horizontal Stabilizers

XKCD.COM · 05 MAY 2026 · SOURCE



It started as a mistake that everyone was afraid to admit to, and then it stuck because removing it 'looks silly.'.

FRED TALKS

6th May 2026

CONTENTS

The future of enterprise software

AARON LEVIE · 28 JAN 2026 · SOURCE

One of the biggest topics right now in the tech world is what does the future of software look like when AI agents are doing a substantial amount of work in the enterprise. Does software get compressed and simply become a database that agents leverage? Do we use AI agents to code all our own personalized enterprise software in the future? Do startups or incumbents win this new battle? How big are software markets in a world of agents using these tools? And much much more.

It's impossible to try and tackle all of these questions sufficiently in one sitting, but I figured I'd lay out a bit of a roadmap for what enterprise software looks like in the future.

Why companies buy software

Firstly, we should all get on the same page about why companies use and buy enterprise software in the first place. You can think of enterprise software as a codification of a company's processes. These are the parts of your company that you *don't* want to change spontaneously, because they provide the structure for your workflows and operations to keep your business running, or allow you to do the other work that really matters.

A large car company decides to procure an ERP system because it's the backbone for complex global operations where failures could mean far more than the cost of the system. It's responsible for keeping track of tens or hundreds of billions in revenue, and the company wants to ensure that every single transaction that they make goes through the exact same way every single time, with five 9s of uptime and reliability, and with an output that they can get accepted and approved by the auditors. Jobs are on the line, all the way up to the CEO, based on how well this system performs.

And the reason that companies choose to use software vendors that specialize in certain domains instead of just building this technology themselves can best be understood by the concept of "core" vs. "context," popularized by Geoffrey Moore. The core stuff is what makes your company unique (like your core product, your talent, your culture, and so on), and the context is absolutely necessary, but largely non-differentiating between firms (think all of your systems of record like payroll, IT ticket responses, ERP systems, content management platforms, and so on).

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

For software that handles your company's context activities, your customer will rarely notice when these things work great, but they'll certainly notice the downstream impacts when they don't. This is why you "rent" these services from software providers that do this as their day job for thousands or millions of other companies. And incidentally, much of this software helps define these underlying workflows in your organization so you don't have to -- so each company doesn't have to reinvent the wheel each on how to handle HR, IT ticketing, and so on.

Deterministic vs. non-deterministic systems

Ok, but if an enterprise is filled with 100X more AI Agents in the future, won't that eventually consume all the use-cases that software did previously?

If you take the analogy of the industrial world, software is like the machinery in a manufacturing plant; and agents are like the people that operate that machinery. Just like machinery in a plant, in deterministic software, you want things to happen the same way, every time: you want data that goes into your ERP system to be calculated based on the business rules you've setup; you want security permissions you've set for accessing critical files to behave the same way every time, and not leak information between users; and you don't want an HR system probabilistically generating new reporting structures for employees on every load.

Conversely, you want *non-deterministic* systems (in this case AI agents) for the things that people have always tended to do in these systems. An agent filling out notes from a user's client meeting in a CRM system, generating a sales presentation for a new customer, answering a customer support inquiry with the best products to leverage, doing research or providing analysis on a set of enterprise documents and data for an important decision, and so on. As a result, the non-deterministic and deterministic elements of software end up working in a complementary fashion.

In contrast to some of the public discussion, I'd argue that in a world of 100X more AI agents than people in an enterprise, the value of the systems of record and tools agents will use will go up, not down. Because in this new world, software provides the guardrails on which agents can operate successfully within an enterprise, and gives them the underlying tools to use to be more productive themselves and work alongside people.

For instance, if you ask an AI agent to generate 10X the number of outbound email and marketing messages, where will all those agents find the data to work with, and where will the leads end up going to have a human sales rep act on them? Almost certainly some form of a CRM system. Or when an AI Agent processes reviews invoices and transactions, where will they

enter this data when they're done to kick off a supply chain workflow? Again, an existing or reinvented ERP system. As long as we have humans and agents interacting with one another, there will need to be deterministic software that bridges these interactions seamlessly.

And as a result of all of these new agentic users within our software systems, the size of enterprise software markets are going to increase massively.

Unconstrained software markets

Traditionally, software companies have been stuck within the constraints of existing IT budgets, which tend to tap out around 3-7% of a company's revenue. This creates an inherent upper limit for what the budget can be for software, which translates into the total addressable market of various technology categories. Now, with AI Agents, the software is actually bringing along the work with the software, which means the budget software players are going after is the total spend that goes into doing that work in the company, not just the tech to enable it. This inevitably leads to a substantial increase in TAM for most software categories whose markets were artificially held back in size previously.

For instance, a contract management vendor just a few years ago was limited by the number of attorneys within an organization that needed to manage and review contracts. However, in a world of AI agents, that cap no longer exists, where you might have agents running continuously processing and reviewing contracts and providing extend legal services for any company. The size of the legal services market in the US is somewhere around $400B, nearly 50-100X larger than the corresponding software categories in legal space. You can easily see how AI agents making the legal industry meaningfully more productive could capture multiple percentage points of that $400B in spend, thus multiplying the size of the legal software market overnight.

The same analogy holds for almost any software category that was artificially constrained by a customer's headcount, or limited software spend, previously. We're already seeing this in spaces like coding, where the AI agents in coding are generating orders of magnitude more revenue for the new coding startups than the prior IDE categories generated. And we can expect this same dynamic for most other areas of knowledge work over time, from healthcare and financial services to legal and marketing. Tools that bring along work will generate far more revenue than the prior era of tools used to.

The opportunity ahead for software

Now, the big open question is who wins in this technology wave. If we have learned anything from Clayton Christensen's Innovator's Dilemma framework, we know that incumbents tend to lose in markets when there's a new tech or business model innovation that is unattractive to

↩

3. In theory, at least. In practice it's always better to be useful (again, in the sense of "delivering shareholder value").

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are here and here.

↩

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

2. Lose the habit of worrying about if you're being treated "fairly". Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won't look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You'll also look like someone who's paying a lot of attention to what their manager's actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you'll eventually find yourself in one of the staff engineer archetypes. However, it probably won't be the one you're "aiming for". The whole point of being a staff engineer is that you're willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role[5]. At the time, the "staff engineer" role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they've built a deep relationship of trust with their organization's leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don't have the formal "staff" title at all, while others have "staff" as a fairly early rung on the ladder and a panoply of "senior staff", "senior principal staff", and so on roles above it. Like all "staff engineer" discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

   ↩

2. Impressing your VP's trusted lieutenants can actually be a good way to build trust in the medium-term, but you'd better hope you've built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

them because it represents less revenue or profitability. Conversely, they tend to hold their own when the new business model or technology is supportive of their existing business, or perhaps even enhances it. Ultimately, in AI, we'll likely see examples of both outcomes.

Some existing software categories, like customer support or coding, are changing so rapidly that many incumbents will be unable to change their business models or technology stacks fast enough to respond to the growing demands of the new market, even if getting to the other side would be economically attractive. Similarly, as Jaya Gupta and Ashu Garg outlined in their piece on context graphs, there are many new categories of agent-native software that will need to be created for the first time because no existing vendor logically can capture the way the agent needs to do its work. This will create tremendous opportunities for new startups to build AI agents within these categories, and we're seeing new ones crop up every day.

Conversely, there are a number of existing SaaS players are in a natural position to power agents in their relevant work categories, especially those that have the natural context necessary for AI agents to be successful in automating work. Unsurprisingly, I'd expect there to be a correlation between the strength of a software company's moat and the amount of data they house, how complex the underlying workflows are, the number of connections there are with other systems, how intertwined a human-in-the-loop element is to the underlying agentic workflow, and how naturally AI agents can be plugged into these workflows (either as native users or via API in external agentic products).

Of course, this will mean that the business model of software (new upstarts and incumbents) will have to evolve over time. Today, the vast majority of SaaS products charge on a per-seat basis, which generally corresponds to most of the usage that the software sees today by its end users. But in a world where AI agents do most of the interaction and work on software, enterprise systems will have to evolve to support more of a consumption and usage-based model over time. AI agents don't cleanly fit as seats on software, because any given AI agent can do a varied amount of work within a system (e.g. you could have 1 agent doing a billion things or a billion agents doing one thing).

Inevitably, we'll see a mix of both outcomes. Users (or people) will continue to be seats within software, and AI agents that represent extended usage will be monetized through consumption. We're already seeing this play out in AI-native coding products and other platforms where there's an end-user seat component mixed with a consumption model on top. Obviously depending on the human vs. agent value proposition mix, the revenue stream will slide between these two ends of the continuum.

Ultimately, there's still plenty that needs to play out before we fully see what the future holds for enterprise software. But it will certainly be the most dynamic and exciting period we've ever seen in software history.

Evolution and Scale of Uber's Delivery Search Platform

DIVYA NAGAR · 05 DEC 2025 · SOURCE

Featured image for Evolution and Scale of Uber's Delivery Search Platform

Introduction

Search is a primary discovery funnel for Uber Eats: a large share of orders start with people typing into the search bar to find stores, dishes, and grocery items. Strong search directly translates into higher conversion, better basket quality, and faster time-to-order—especially for long-tail queries, new or seasonal items, and multilingual markets. When search misses intent, people bounce or fall back to browsing. When it understands intent, they find what they want in seconds.

Traditional search stacks begin with lexical matching, which is fast and effective when queries exactly match document text. But real queries are challenging—synonyms ("soda" versus "soft drink"), typos ("mozzarela"), shorthand ("gf pizza"), language mix ("pan" meaning bread in Spanish, but a container for cooking in English), and context ("apple" the fruit vs the company). Lexical methods see strings, not meaning, and aren't suitable for such queries, producing bad search results.

Semantic search shifts from matching words to matching meaning. It encodes queries and documents into vectors in the same space, so semantically similar things are close—even without keyword overlap. When properly used, it delivers a search experience that better captures someone's intent across verticals (stores, dishes, items) and languages.

- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned "minders" (often current staff engineers) who will ghost-lead the project through you[2].
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The "solver" and "right hand" archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of "aiming for" a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don't get the support needed to succeed) that's their manager's problem, not theirs[3]. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can't be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn't possible to conjure useful work from thin air. That's actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don't realize is that all senior management and executive leadership roles are unfair too, in the same way. That's just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather[4]. "Staff engineer" is the first engineering role where you are held largely responsible for outcomes you don't control.

Developing a "staff engineer mindset" thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself "is this useful to the company" (and answering correctly).

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · SOURCE

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *Staff engineer archetypes*. He argues that the "staff engineer" title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the "staff engineer archetypes" and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist[1]. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the "team lead" archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard "to be a writer". You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - "good work" here means "deliver shareholder value" - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!

Establishing semantic search at scale for industry applications is more than training a model, but instead a whole tech stack including model deployment, ANN (Approximate Nearest Neighbor) index serving at scale, monitoring, version control, and so on. This blog walks you through how Uber Eats built this system, reviewing challenges in this process and the solutions we took.

Architecture

The general semantic search model architecture is shown in Figure 1. We adopted the classic two-tower structure so that the query and document embedding calculation can be decoupled. The query embeddings are calculated via an online service in real time, while the document embeddings are calculated in a batch manner via offline scheduled services. When training the model, we used the MRL (Matryoshka Representation Learning)-based infoNCE loss function so that a single model could output various embedding dimensions, and downstream tasks could choose the optimal embedding dimension according to their own size versus quality tradeoffs.

The latest LLMs (QWEN®) are used as the backbone embedding layer inside two towers to leverage its world knowledge and cross-lingual abilities. We further finetuned them using Uber Eats in house data to adapt the embedding space to our application scenarios. Currently, this single embedding model supports the content embedding task for all Uber Eats verticals and markets.

Image

Figure 1: Model training architecture.

Model Training and Inference

Our model is a two-tower deep neural network built on a Qwen backbone. We use PyTorch™ with Hugging Face Transformers® for model development and Ray™ for distributed training orchestration. For large-scale training, we use PyTorch's DDP (Distributed Data Parallel) together with DeepSpeed® (ZeRO-3), which shards the optimizer, gradients, and model parameters to handle the immense size of the Qwen model. This setup, combined with mixed-precision training and gradient accumulation, allows us to train on hundreds of millions of data points efficiently. Each successful training run generates versioned artifacts—a unique ID for the query and document models (*query_model_id*, *doc_model_id*), and a shared team-specific ID called two-tower-embedding-id (*tte_id*)—which are meticulously tracked for reproducibility and deployment.

Given the scale of our document corpus, offline inference is critical. To manage costs, we embed documents at the feature level (for example, store or item) and then join these embeddings back to the full catalog of billions of candidates before indexing. The resulting embeddings are stored in stable feature store tables, keyed by entity IDs, making them readily available for both retrieval and ranking models. These embeddings are used to build our search index, which are supported by HNSW graphs and store both non-quantized (float32) and quantized (int8) vector representations to optimize for various latency and storage budgets.

Image

Figure 2: End-to-end training inference workflow.

Development and Deployment Challenges

We tackled key challenges in scaling our semantic search system. For example, we had to balance retrieval accuracy with infrastructure and compute costs through careful tuning of ANN parameters, quantization, and embedding dimensions.

To achieve productionization and reliability, we had to ensure safe, automated, and rollback-capable index deployments with strong validation, versioning, and real-time consistency checks in production.

Quality and Cost

We use Apache Lucene® Plus as our primary indexing and retrieval system. To manage the scale and diverse nature of the data, we maintain separate indices for each vertical—OFD (restaurants) and GR (grocery/retail).

Storing and serving large-scale indexes can be expensive, in terms of infrastructure costs and query latency. To optimize this tradeoff while maintaining strong retrieval quality, we explored several mechanisms to balance cost, latency, and performance. In particular, we ran controlled experiments varying three key factors:

- **Search parameter K**: controls the number of nearest neighbors retrieved per shard in ANN search. By carefully choosing k per shard—guided by data distribution statistics—you can maintain the overall retrieval cohort size and preserve result quality.
- **Quantization strategy**: comparing non-quantized float32 embeddings against compact int7 SQ (scalar-quantized) representations.

Zheng Liu is a Staff Software Engineer on Uber Eats Search team, where he focuses on improving search experience using advanced deep learning and large language models. He's a major contributor to Uber Eats semantic search and generative ranking models.

Jiasen Xu

Jiasen Xu

Jiasen Xu is a Sr. Software Engineer on Uber's Search Platform team, where he leads the adoption of vector search technologies across the company. His work focuses on advancing large-scale similarity search through expertise in ANN algorithms, indexing libraries, and vector database optimization.

Bo Ling

Bo Ling

Bo Ling is a Sr. Staff Software Engineer on Uber's AI Platform team. He works on NLP, large language models, and recommendation systems. He's the lead engineer for embedding models and LLM on the team.

Haoyang Chen

Haoyang Chen

Haoyang Chen is a Sr. Staff Software Engineer on Uber Eats Search team, where he leads efforts in semantic search and generative recommendation to enhance the search experience on Uber Eats.

Posted by Divya Nagar, Zheng Liu, Jiasen Xu, Bo Ling, Haoyang Chen

Category:

Engineering

Data / ML

Uber AI

What makes this system stand out is its production-first design: it automatically retrains, rebuilds, and refreshes search indexes every two weeks, ensuring that results stay fresh and reliable as data changes daily. With a robust blue/green deployment strategy, strong validation gates, and efficient index optimization, we can safely roll out new models without disrupting live traffic.

Overall, this platform demonstrates how thoughtful engineering—combining strong modeling, reliable infrastructure, and cost-aware optimization—can deliver a search experience that feels smarter, faster, and more intuitive for users around the world.

Stay up to date with the latest from Uber Engineering—follow us on LinkedIn for our newest blog posts and insights.

Divya Nagar

Divya Nagar

Divya Nagar is a Staff Software Engineer on the Feature Engine team at Uber, where she leads the development of the Vector Store, the data plane powering embedding use cases across both generative AI and CoreML applications.

Zheng Liu

Zheng Liu

- **Embedding dimension**: using different Matryoshka-cut representations generated via MRL (Matryoshka Representation Learning), such as *emb_256* and *emb_512*.

Here's how we designed search parameters and quantization:

Image

Table 1: Impact of shard-level k and quantization on recall and cost savings.

We also used MRL (Matryoshka representation learning) to reduce the dimension of the embeddings to optimize the latency and storage costs further. MRL solves this by training a single model whose embeddings can be cut at different lengths. Shorter versions of the embedding capture coarse information quickly, while longer versions add finer details for higher accuracy. This gives us the flexibility to trade off speed versus quality at inference time, without retraining separate models.

In our application, we chose the following vector dimensions with equal weights: [128, 256, 512, 768, 1024, 1280, 1536]. In our offline evaluation results, we observed that the MRL provides strong performance in dimension reduction.

Image

Table 2: Dimension reduction performance difference from MRL.

Lowering shard-level *k* from 1,200 to around 200 yielded a 34% latency reduction and 17% CPU savings in our experiments, with negligible impact on recall. On top of that, applying scalar quantization (int7) further cut compute costs, reducing latency by more than half compared to fp32 while maintaining recall above 0.95. MRL added another layer of efficiency by enabling us to serve smaller embedding cuts (256 dimensions) with almost no loss (< 0.3% for English and Spanish) in retrieval quality compared to full-size embeddings, while also reducing storage costs by nearly 50%. Together, these optimizations significantly lowered the cost and latency of serving large-scale indexes without any significant compromising in retrieval quality.

Beyond quantization and k tuning, our vector index is paired with locale-aware lexical fields and boolean pre-filters—including *hexagon*, *city_id*, *doc_type*, and *fulfillment_types*—that run before the ANN search. These pre-filters dramatically shrink the candidate set up front, ensuring low-latency retrieval even against a multi-billion document corpus. Once the ANN search

identifies the top-k candidates per vertical, we optionally apply a lightweight micro-re-ranking step, using a compact neural network, to further refine results before handing them off to downstream rankers.

Productionization and Reliability

Uber data changes continuously. To keep results fresh, our Delivery semantic search runs a scheduled end-to-end workflow that updates the embedding model and serving index in lockstep on a biweekly cadence. Each run trains or fetches the target model, generates document embeddings, and builds a new Lucene Plus index without changing the live read path.

To enable seamless model refreshes without disrupting live traffic—and to provide rollback flexibility when issues arise—we adopted a blue/green pattern at the index column level. Each index maintains two fixed columns, *embedding_blue* and *embedding_green*, with the active model version mapped to one of them via a lightweight configuration. During each refresh, a new index snapshot is built containing both columns: the active column is preserved as-is, while the inactive column is repopulated with fresh embeddings. This design allows us to safely switch between old and new versions when needed.

During each refresh, we rebuild a full index snapshot that contains both columns (*embedding_blue* and *embedding_green*). Even though only the inactive column is intended to change, the build re-materializes the active column as well, so a bug or bad input could silently corrupt what's currently serving. To protect production, we gate with automated validations that run before any deployment: they block the flow if inputs are incomplete, if the carried-forward (active) column diverges significantly from the current prod index, or if the newly refreshed column underperforms.

Concretely, our pipelines run three checks.

- **Completeness:** verify document counts (and key filtered subsets) in the index match the source tables used to compute embeddings—fail fast on drift or partial ingestion.
- **Backward-compatibility:** ensure the unchanged column is byte-for-byte equivalent between the pre-refresh and post-refresh index (for example, at T1 compare *embedding_blue* across old and new builds if *embedding_blue* is currently serving. Any mismatch ends the run.
- **Correctness:** deploy the newly built index to a non-prod environment, replay real queries at a controlled rate, and compare recall against the current production index. These checks catch issues like data corruption during indexing, bad distance settings, or upstream feature regressions, ensuring the new index is at least as good as production before we make a switch.

We also considered maintaining two separate blue and green indexes instead of two columns within a single index. While that approach would simplify reliability—since we wouldn't need to touch the production index during a refresh—it'd significantly increase storage and maintenance costs. Given that trade-off, we built a mechanism that lets us safely operate with one index containing two columns.

Image

Figure 3: Model<>index deployment and serving flow.

Even with strong build-time and deploy-time checks, there's still a risk of human error or bugs. For example, a wrong model-to-column mapping or an index built against a different-than-expected model. To guard against this, we added serving-time reliability checks. Reliability of the production system was a significant challenge during the development process.

To make the system more reliable and prevent outages, we deploy the new index to production, then roll out the model gradually. The index stores the model ID in its metadata for both columns. On sampled requests, the service verifies that the model used to generate the query embedding matches the model ID recorded on the active index column. On any mismatch, we emit a compatibility-error metric and log full context. If errors remain non-zero over a short window, an alert fires and we automatically roll back the model deployment to the previous version (the index remains unchanged). This protects against version mix-ups and config mistakes without adding latency or extra hops to the read path.

Conclusion

We built a scalable, multilingual search system for Uber Eats that powers discovery across restaurants, grocery, and retail with speed and precision. Our approach brings together advanced language models and smart architecture choices—like Matryoshka embeddings and Lucene Plus integration—to balance quality, cost, and latency at a global scale.